Project Review

Team 6 - Paradigm

Members

Santórum Sandoval Thais Yetsalem

Toapanta Orejuela Adrian Michael

Vargas Pérez César Alexander

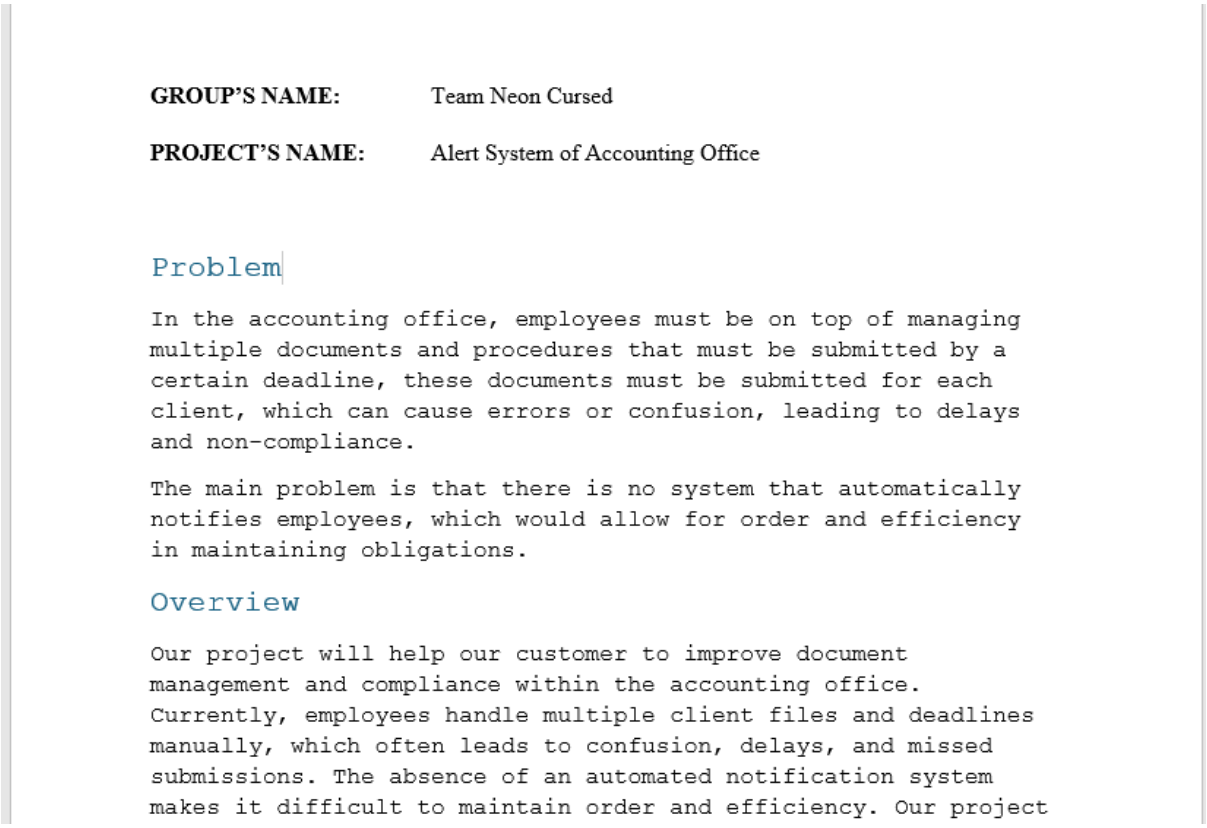
Reviewers: Toapanta Orejuela Adrian Michael
Vargas Pérez César Alexander
Scribe: Santórum Sandoval Thais Yetsalem

Order / Artifact		Evaluation/ 10	Observations
1. Overview		10	Quite solid, the information goes straight to the point, is precise, and clearly explains what the proposed system aims to achieve.
2. SRS		9	There are several dates and small sections that have not been updated and were left as they were found in the document template.
Features	3. Prioritized Features	8.1	Identity and RUC validation is mentioned, but within the prototype there is only a login that is not necessarily related to what is described.
	4. Tasks	8.3	The document is an excellent basis for OOP design, but the "Task" as a business concept in the accounting office needs to be detailed with specific accounting information and alert rules, also it needs to focus on the tasks rather than on the concepts that helped to develop them.
	5. Plan	9.3	There are five features in the planning, which does not match the six detailed in the requirements document as well as in the SRS.
Design	6. Architecture	10	It´s following the architecture.
	7. Use Case	9	To improve the diagram, generic use cases like "Manage Client" should be broken down into specific actions. Additionally, UML relationships (<<include>> and <<extend>>) should be added to clarify dependencies between functionalities. Finally, security use cases like "Login" are essential.

	8. Class Diagram	8	In the diagram and code have different data type (in UML phone is an int and in the program is a String). And some relations between classes are wrong.
	9. Mockups	8	It should have been a bit more detailed, indicating each option instead of just copying the initial menu.
Prototype: CMD	10. Functionality validation	8.1	The lack of validations and sound programming practices has resulted in the possibility of using historical dates within the system.

EVIDENCE:

1. Overview



GROUP'S NAME: Team Neon Cursed

PROJECT'S NAME: Alert System of Accounting Office

Problem

In the accounting office, employees must be on top of managing multiple documents and procedures that must be submitted by a certain deadline, these documents must be submitted for each client, which can cause errors or confusion, leading to delays and non-compliance.

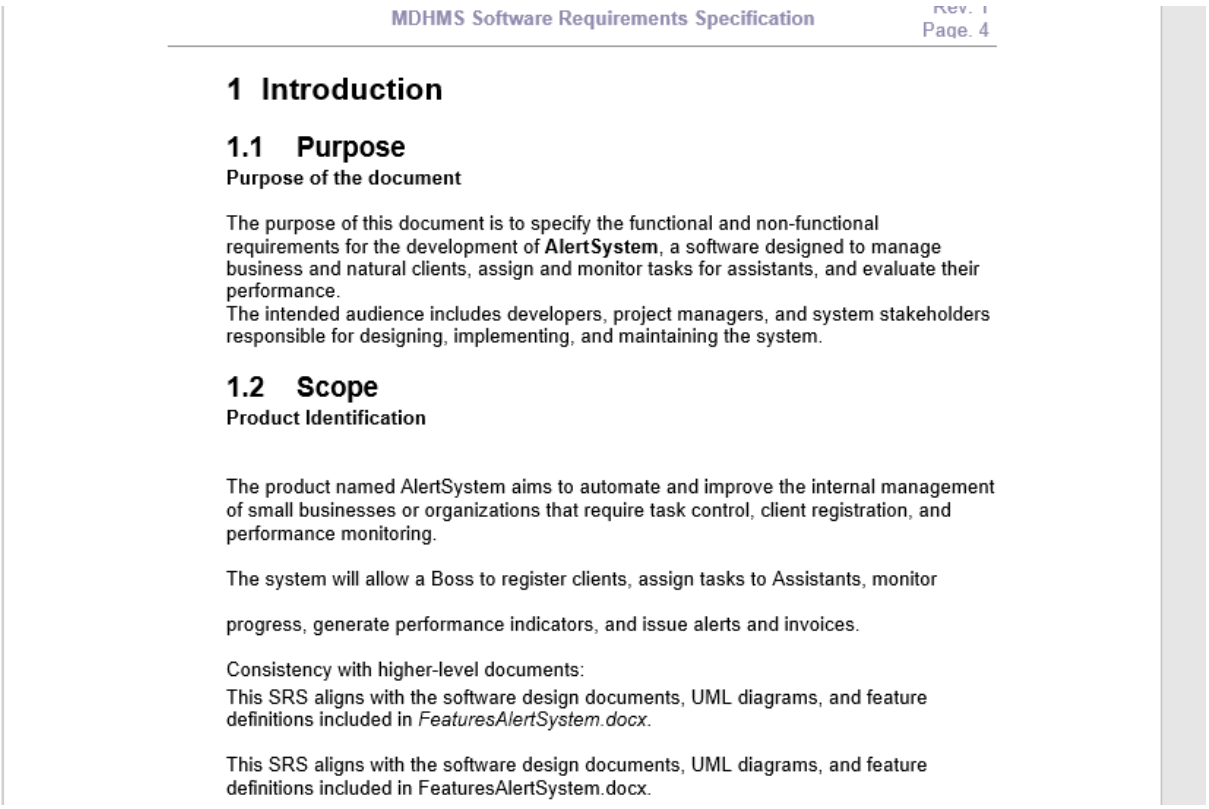
The main problem is that there is no system that automatically notifies employees, which would allow for order and efficiency in maintaining obligations.

Overview

Our project will help our customer to improve document management and compliance within the accounting office. Currently, employees handle multiple client files and deadlines manually, which often leads to confusion, delays, and missed submissions. The absence of an automated notification system makes it difficult to maintain order and efficiency. Our project

Quite solid, the information goes straight to the point, is precise, and clearly explains what the proposed system aims to achieve.

2. SRS



MDHMS Software Requirements Specification Rev. 1
Page. 4

1 Introduction

1.1 Purpose

Purpose of the document

The purpose of this document is to specify the functional and non-functional requirements for the development of **AlertSystem**, a software designed to manage business and natural clients, assign and monitor tasks for assistants, and evaluate their performance.
The intended audience includes developers, project managers, and system stakeholders responsible for designing, implementing, and maintaining the system.

1.2 Scope

Product Identification

The product named AlertSystem aims to automate and improve the internal management of small businesses or organizations that require task control, client registration, and performance monitoring.

The system will allow a Boss to register clients, assign tasks to Assistants, monitor

progress, generate performance indicators, and issue alerts and invoices.

Consistency with higher-level documents:
This SRS aligns with the software design documents, UML diagrams, and feature definitions included in *FeaturesAlertSystem.docx*.

This SRS aligns with the software design documents, UML diagrams, and feature definitions included in FeaturesAlertSystem.docx.

There are several dates and small sections that have not been updated and were left as they were found in the document template.

Features

3. Prioritized Features

- Josue Rojas

Shall we should

1. **User Management**

   Registration and administration of clients (individuals, companies)

   Identity and ruc verification.

2. **Task Management**

   Creation, updating, and deletion of tasks.

   Assignment of tasks to assistants.

   Tracking of task status.

3. **Performance Evaluation**

   Each assistant has a set of evaluated tasks and a performance record.

   Objectively evaluate assistant performance based on task completion.

4. **Alert System**

   Generation of automatic alerts to detect overdue and pending tasks.

5. **Invoice Management for Completed Tasks**

   Automatic creation of invoices for completed tasks.

6. **Activity Management by Manager and Assistant**

   The manager manages: Tasks, deals, people.

   The assistant executes: assigned tasks.

Identity and RUC validation is mentioned, but within the prototype there is only a login that is not necessarily related to what is described.

4. Tasks

Core OOP Concepts for the Project

1. Class
   A blueprint that defines the properties and behaviors (attributes and methods) of objects.
   Example: Task, User, Manager, Assistant, Invoice, Alert.

2. Object
   An instance of a class that represents a specific entity in the system.
   Example: A specific task like “Prepare Tax Report” or an assistant named “John Doe”.

3. Encapsulation
   The concept of hiding internal data and only exposing necessary functionality through methods (getters and setters).
   Example: Task status or deadline are private attributes accessed only through getStatus() or setStatus().

The document is an excellent basis for OOP design, but the "Task" as a business concept in the accounting office needs to be detailed with specific accounting information and alert rules, also it needs to focus on the tasks rather than on the concepts that helped to develop them.

Design

5. Plan

week 1 (13 - 19 October 2025)

1. **Task Management**

   Creation, updating, and deletion of tasks.

   Assignment of tasks to assistants.

   Tracking of task status.

week 2 (20 - 26 October 2025)

2. **Performance Evaluation**

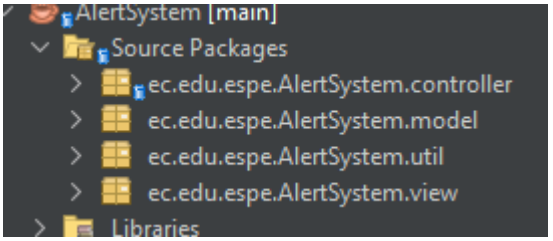
   The system calculates simple indicators such as:

   Tasks completed: Number of tasks finished by the assistant.

   On-time completion: Percentage of tasks completed before the deadline.

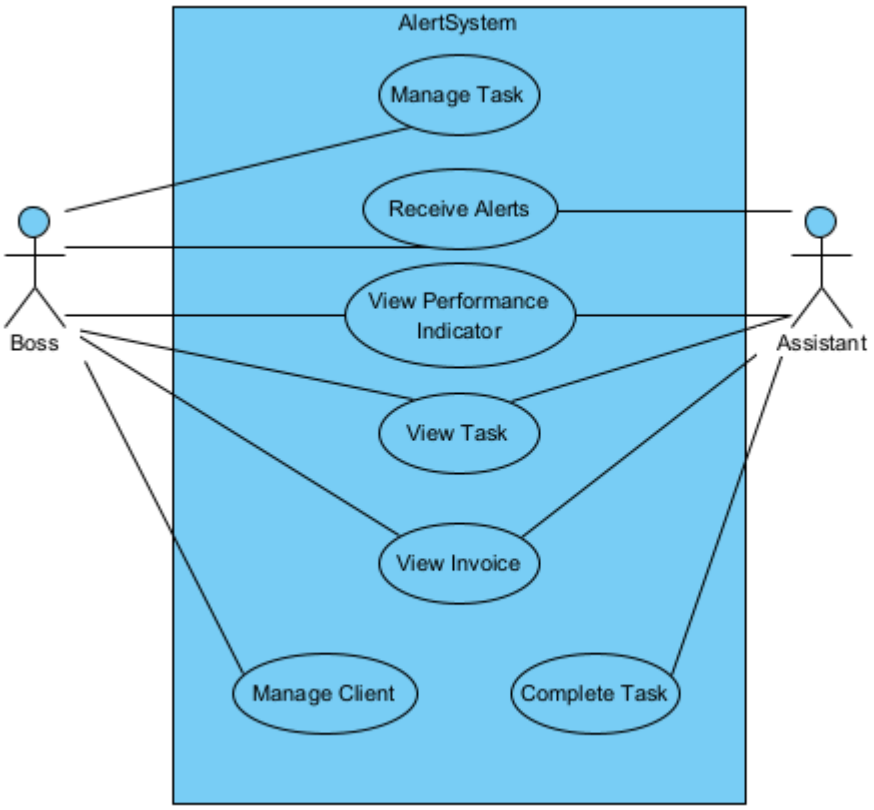
There are five features in the planning, which does not match the six detailed in the requirements document as well as in the SRS.

6. Architecture



It´s following the architecture.

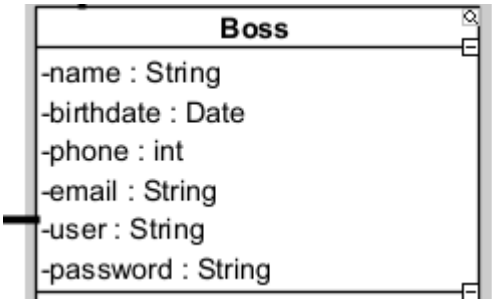
7. Use Case



To improve the diagram, generic use cases like "Manage Client" should be broken down into specific actions. Additionally, UML relationships (<<include>> and <<extend>>) should be added to clarify dependencies between functionalities. Finally, security use cases like "Login" are essential.

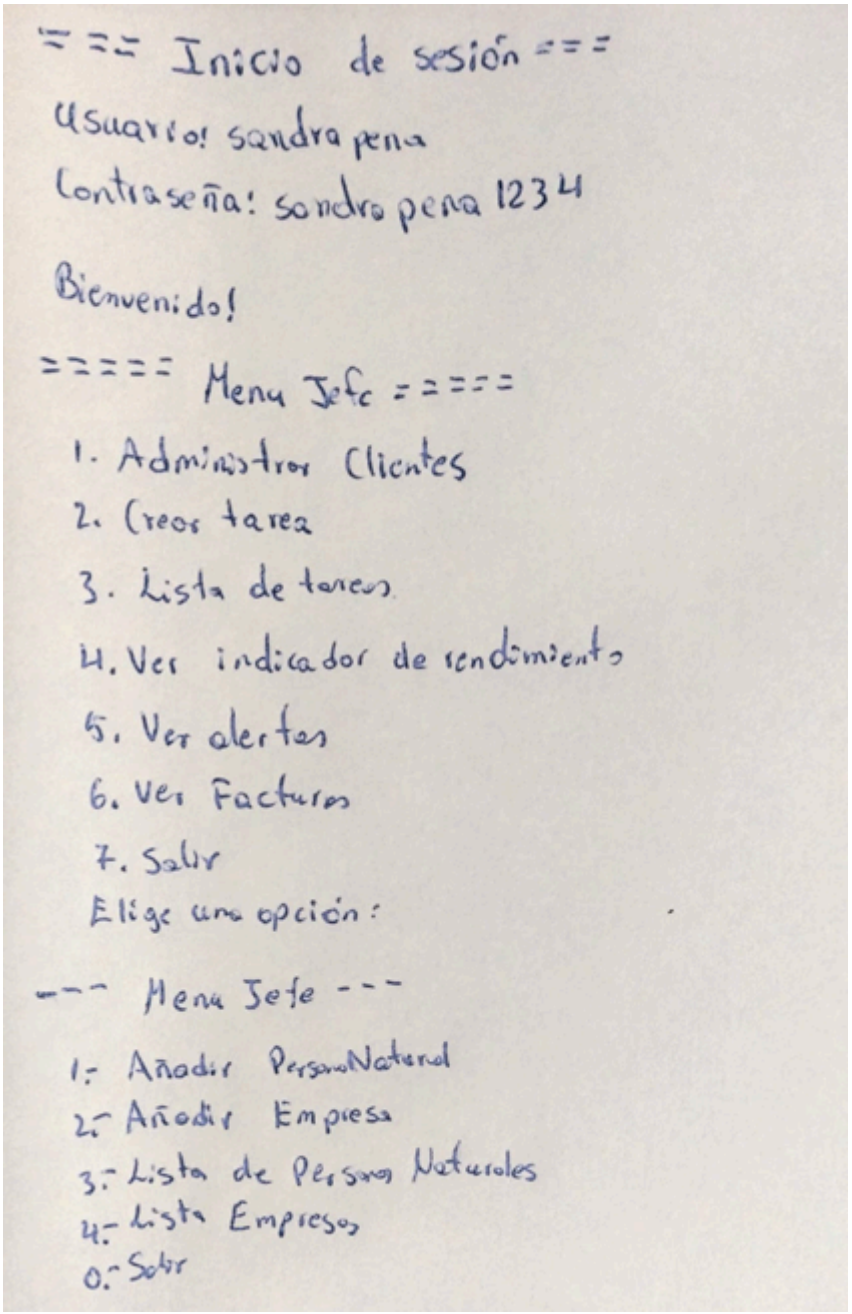
8. Class Diagram

```
public class Boss {
    private String name;
    private Date birthdate;
    private String phone;
    private String email;
    private String user;
    private String password;
```



In the diagram and code have different data type (in UML phone is an int and in the program is a String)

9. Mockups



It should have been a bit more detailed, indicating each option instead of just copying the initial menu.

Prototype:
CMD,
10. functionality validation

```
        String pass = sc.nextLine();

        if (controller.loginBoss(user, pass)) {
            System.out.println("\nBienvenido!");
            BossView bossView = new BossView();
            bossView.showBossMenu();
            loggedIn = true; // salir del bucle
        } else if (controller.loginAssistant(user, pass)) {
            System.out.println("\nBienvenido!");
            AssistantView assistantView = new AssistantView(user);
            assistantView.showMenu();
            loggedIn = true; // salir del bucle
        } else {
            System.out.println("\nCredenciales no válidas. Inténtelo de nuevo...\n");
```

```
=== Inicio de sesi□n ===
Usuario: asdjbasd
Contrase□a: asdasndasbkdas

Credenciales no v□lidas. Int□ntelo de nuevo...

=== Inicio de sesi□n ===
Usuario:
```

The lack of validations and sound programming practices has resulted in the possibility of using historical dates within the system.

GRADES

1. Overview 10 / 10
2. SRS 9 / 10

Features

3. Prioritized Features 8.1 / 10
4. Tasks 8.3 / 10
5. Plan 9.3 / 10

Design

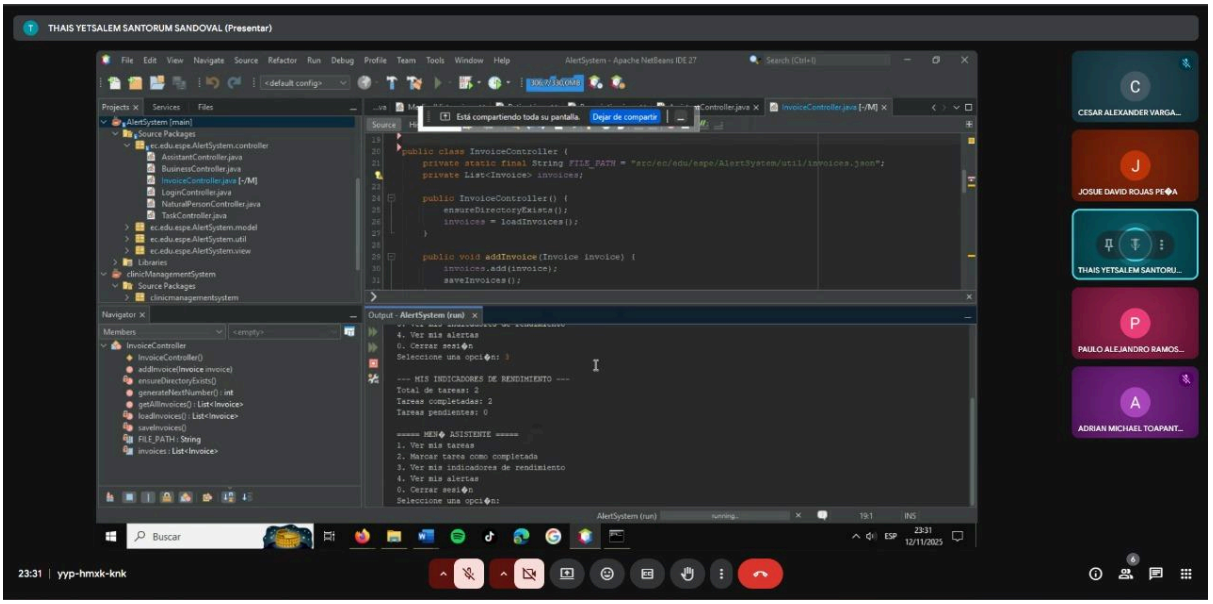
6. Architecture 10 / 10
7. Use Case 9 / 10
8. Class Diagram 8 /10
9. Mockups 8/10

Prototype:

CMD,
10. functionality validation 8.1

AVERAGE: 8.97/10

Evidence of the meeting:



Evidence of the clean code review:

Observations.

- in class BusinessController
  DataManager.

  There are many unnecessary comments above each method: the method name is already clear, so the comment can be omitted because it does not add anything.

- in class BusinessController
  TaskController
  AssistantController
  LoginController

  There are the license URL addresses in the first two lines of the code: this information is unnecessary, and in class we were told that we must remove it.

- in class BossController. BossView

  There's messy indentation starting from line 33.

Recomendations.

- Review the code to ensure clean and consistent indentation.
- Remove comments that do not contribute anything due to the obviousness of the names - whether they are objects, class or functions that are already self explanatory.
- Fix the issue within the view package where more than one 'main' exists, and move any unnecesary files to their appropiate packages.